

# Projet Nadhif - Conception

## Application de Gestion des Plaintes pour la Collecte des Déchets

Epic Nadhif Bouira

Novembre 2025

### Table des matières

|          |                                                              |           |
|----------|--------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                          | <b>3</b>  |
| 1.1      | Vue d'ensemble du système . . . . .                          | 3         |
| 1.2      | Technologies utilisées . . . . .                             | 3         |
| <b>2</b> | <b>Module Géographique</b>                                   | <b>4</b>  |
| 2.1      | Détection Automatique de la Région . . . . .                 | 4         |
| 2.1.1    | Description . . . . .                                        | 4         |
| 2.1.2    | Diagramme de séquence . . . . .                              | 4         |
| 2.1.3    | Algorithme . . . . .                                         | 5         |
| 2.2      | Gestion des Régions Géographiques . . . . .                  | 6         |
| 2.2.1    | Description . . . . .                                        | 6         |
| 2.2.2    | Diagramme de séquence . . . . .                              | 6         |
| 2.2.3    | Algorithme : Création de région . . . . .                    | 7         |
| 2.2.4    | Algorithme : Modification de région . . . . .                | 7         |
| 2.2.5    | Algorithme : Suppression de région . . . . .                 | 8         |
| 2.3      | Consultation et Filtrage des Plaintes sur la Carte . . . . . | 9         |
| 2.3.1    | Description . . . . .                                        | 9         |
| 2.3.2    | Diagramme de séquence . . . . .                              | 9         |
| 2.3.3    | Algorithme . . . . .                                         | 10        |
| 2.3.4    | Filtres disponibles . . . . .                                | 10        |
| <b>3</b> | <b>Module Citoyen</b>                                        | <b>11</b> |
| 3.1      | Suivi de Plainte . . . . .                                   | 11        |
| 3.1.1    | Description . . . . .                                        | 11        |
| 3.1.2    | Diagramme de séquence . . . . .                              | 11        |
| 3.1.3    | Algorithme . . . . .                                         | 12        |
| <b>4</b> | <b>Module Administratif</b>                                  | <b>13</b> |
| 4.1      | Consultation des Plaintes . . . . .                          | 13        |
| 4.1.1    | Description . . . . .                                        | 13        |
| 4.1.2    | Diagramme de séquence . . . . .                              | 13        |
| 4.1.3    | Algorithme . . . . .                                         | 14        |
| 4.2      | Modification du Statut des Plaintes . . . . .                | 15        |
| 4.2.1    | Description . . . . .                                        | 15        |

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| 4.2.2    | Diagramme de séquence . . . . .                    | 15        |
| 4.2.3    | Algorithme . . . . .                               | 15        |
| 4.3      | Export de Données . . . . .                        | 16        |
| 4.3.1    | Description . . . . .                              | 16        |
| 4.3.2    | Diagramme de séquence . . . . .                    | 16        |
| 4.3.3    | Algorithme . . . . .                               | 16        |
| <b>5</b> | <b>Gestion des Ressources</b>                      | <b>17</b> |
| 5.1      | Gestion des Utilisateurs . . . . .                 | 17        |
| 5.1.1    | Description . . . . .                              | 17        |
| 5.1.2    | Diagramme de séquence . . . . .                    | 17        |
| 5.1.3    | Algorithme . . . . .                               | 18        |
| 5.2      | Gestion des Équipes . . . . .                      | 18        |
| 5.2.1    | Description . . . . .                              | 18        |
| 5.2.2    | Diagramme de séquence . . . . .                    | 19        |
| 5.2.3    | Algorithme . . . . .                               | 20        |
| <b>6</b> | <b>Module Analyse et Intelligence Artificielle</b> | <b>21</b> |
| 6.1      | Validation IA des Images de Plaintes . . . . .     | 21        |
| 6.1.1    | Description . . . . .                              | 21        |
| 6.1.2    | Diagramme de séquence . . . . .                    | 21        |
| 6.1.3    | Algorithme . . . . .                               | 22        |
| 6.1.4    | Types de déchets détectables . . . . .             | 22        |
| 6.2      | Statistiques et Graphiques . . . . .               | 23        |
| 6.2.1    | Description . . . . .                              | 23        |
| 6.2.2    | Diagramme de séquence . . . . .                    | 23        |
| 6.2.3    | Algorithme . . . . .                               | 24        |
| 6.2.4    | Types de graphiques disponibles . . . . .          | 24        |
| <b>7</b> | <b>Modèle de Données</b>                           | <b>25</b> |
| 7.1      | Table utilisateurs . . . . .                       | 25        |
| 7.2      | Table wilayas . . . . .                            | 25        |
| 7.3      | Table regions . . . . .                            | 25        |
| 7.4      | Table equipes . . . . .                            | 25        |
| 7.5      | Table region_equipe . . . . .                      | 26        |
| 7.6      | Table plaintes . . . . .                           | 26        |
| <b>8</b> | <b>Architecture Système</b>                        | <b>27</b> |
| 8.1      | Architecture 3-tiers . . . . .                     | 27        |
| 8.2      | Flux de données . . . . .                          | 27        |
| 8.3      | Sécurité . . . . .                                 | 27        |
| <b>9</b> | <b>Conclusion</b>                                  | <b>27</b> |

# 1 Introduction

Ce document présente la conception détaillée du projet Nadhif - Application de gestion des plaintes pour Epic Nadhif Bouira. Le système permet aux citoyens de signaler des problèmes de collecte de déchets et aux administrateurs de gérer efficacement les régions, équipes et plaintes.

## 1.1 Vue d'ensemble du système

Le système Nadhif est composé de trois modules principaux :

- **Module Citoyen** : Dépôt et suivi des plaintes
- **Module Géographique** : Gestion des régions et détection automatique
- **Module Administratif** : Gestion des ressources et traitement des plaintes

## 1.2 Technologies utilisées

- **Backend** : Supabase (PostgreSQL + PostGIS + Storage)
- **Frontend Mobile** : Flutter (Android/iOS)
- **Frontend Web** : React
- **Cartographie** : Leaflet + Leaflet.draw

## 2 Module Géographique

### 2.1 Détection Automatique de la Région

#### 2.1.1 Description

Cette fonctionnalité détermine automatiquement dans quelle région se trouve une plainte en fonction de sa géolocalisation. Elle utilise l'algorithme point-dans-polygone avec PostGIS pour vérifier si les coordonnées GPS appartiennent à une région Nadhif, à la wilaya de Bouira, ou sont hors zone.

#### 2.1.2 Diagramme de séquence

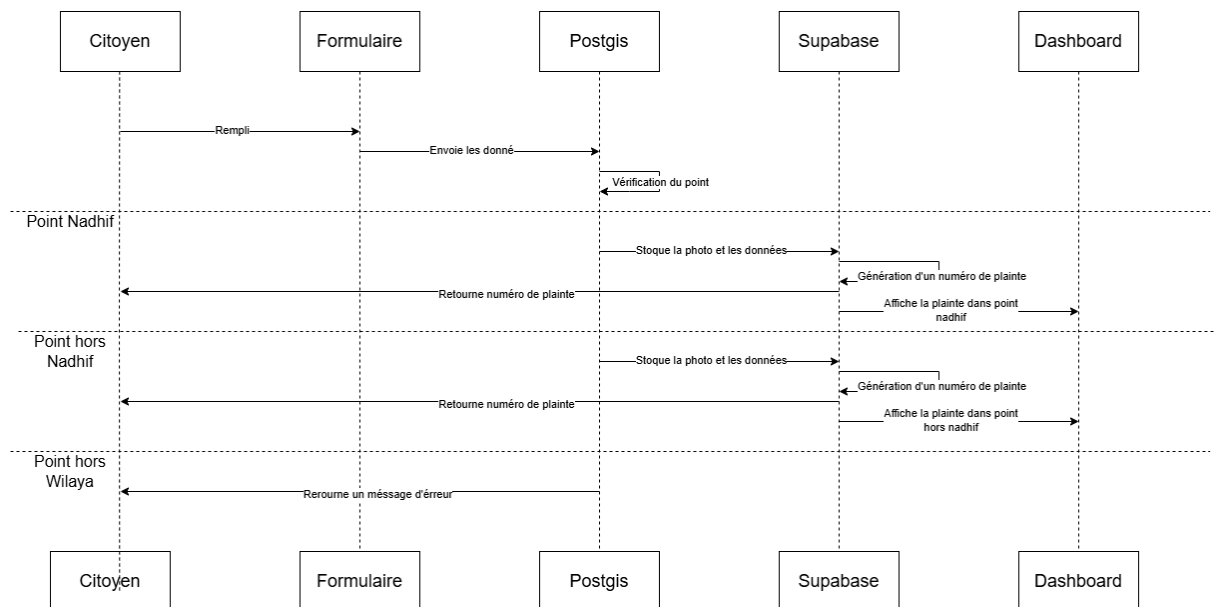


FIGURE 1 – Diagramme de séquence - Détection automatique de la région

### 2.1.3 Algorithme

---

**Algorithm 1** Détection automatique de la région

---

```
1: DÉBUT DETECTERREGIONPLAINT(latitude, longitude, photo, commentaire)
2:   Créer un point géographique avec latitude et longitude
3:   Récupérer le polygone de la wilaya de Bouira
4:   Vérifier si le point est dans la wilaya avec PostGIS
5:   SI le point est HORS wilaya ALORS
6:     Rejeter la plainte
7:     RETOURNER "Plainte hors zone de compétence Nadhif"
8:   Récupérer toutes les régions Nadhif actives
9:   POUR chaque région dans les régions Nadhif FAIRE
10:    SI le point est dans cette région ALORS
11:      Enregistrer la plainte avec regionId
12:      Générer un numéro de plainte
13:      RETOURNER numéro et message de succès
14:    Enregistrer la plainte sans regionId (Point hors Nadhif)
15:    Générer un numéro de plainte
16:    RETOURNER numéro et message "Zone non couverte"
```

---

## 2.2 Gestion des Régions Géographiques

### 2.2.1 Description

Permet aux administrateurs de créer, modifier et supprimer des régions géographiques sur une carte interactive. Les régions sont dessinées sous forme de polygones et validées pour s'assurer qu'elles restent dans les limites de la wilaya de Bouira.

### 2.2.2 Diagramme de séquence

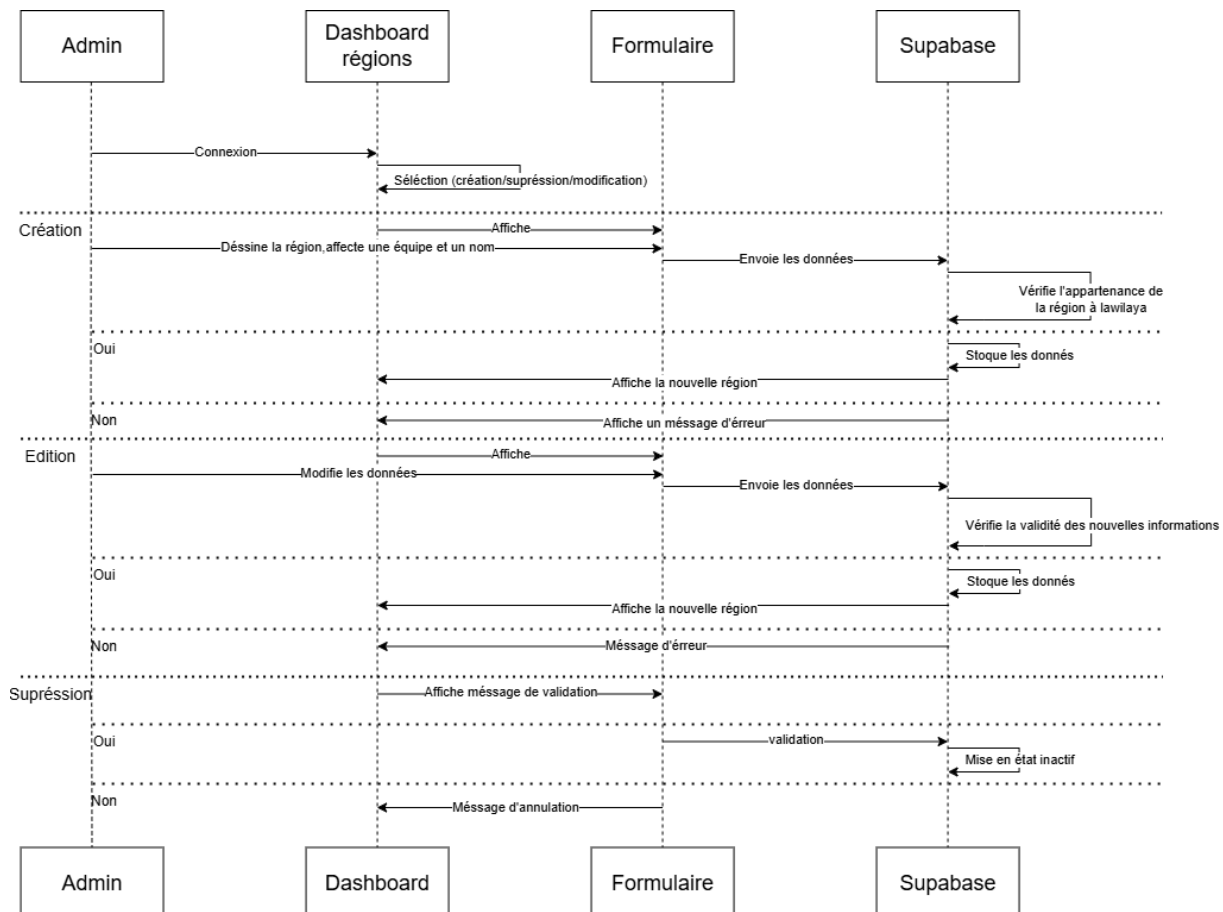


FIGURE 2 – Diagramme de séquence - Gestion des régions géographiques

### 2.2.3 Algorithme : Création de région

---

**Algorithm 2** Créer une région

---

```
1: DÉBUT CREERREGION(listePoints, nomRegion, typeRegion, equipeId)
2:   SI listePoints a moins de 3 points ALORS
3:     RETOURNER erreur "Polygone invalide"
4:   Créer polygone avec listePoints
5:   Récupérer polygone de la wilaya de Bouira
6:   Vérifier si polygone dans wilaya avec PostGIS
7:   SI polygone dépasse la wilaya ALORS
8:     RETOURNER erreur "Région hors limites"
9:   Enregistrer région dans base
10:  SI equipeId fourni ALORS
11:    Créer liaison région-équipe
12:  Afficher région sur carte
13:  RETOURNER succès
```

---

### 2.2.4 Algorithme : Modification de région

---

**Algorithm 3** Modifier une région

---

```
1: DÉBUT MODIFIERREGION(regionId, nouveauxPoints, nouveauNom, nouvelleEquipeId)
2:   Récupérer région avec regionId
3:   SI région n'existe pas ALORS
4:     RETOURNER erreur "Région introuvable"
5:   SI nouveauxPoints sont fournis ALORS
6:     Créer nouveau polygone avec nouveauxPoints
7:     Récupérer polygone de la wilaya de Bouira
8:     Vérifier si nouveau polygone dans wilaya avec PostGIS
9:     SI polygone dépasse la wilaya ALORS
10:      RETOURNER erreur "Nouvelle forme hors limites"
11:    Mettre à jour la géométrie
12:  SI nouveauNom est fourni ALORS
13:    Mettre à jour le nom
14:  SI nouvelleEquipeId est fournie ALORS
15:    Désactiver ancienne affectation équipe
16:    Créer nouvelle affectation équipe
17:  Enregistrer les modifications dans base
18:  Rafraîchir affichage de la région sur carte
19:  RETOURNER "Modification réussie"
```

---

### 2.2.5 Algorithme : Suppression de région

---

**Algorithm 4** Supprimer une région

---

```
1: DÉBUT SUPPRIMERREGION(regionId)
2:   Récupérer région avec regionId
3:   SI région n'existe pas ALORS
4:     RETOURNER erreur "Région introuvable"
5:   Demander confirmation à l'admin
6:   SI admin annule ALORS
7:     RETOURNER "Opération annulée"
8:   Mettre région en état inactif (soft delete)
9:   Désactiver affectation équipe
10:  Masquer région sur carte
11:  RETOURNER "Région désactivée avec succès"
```

---



## 2.3 Consultation et Filtrage des Plaintes sur la Carte

### 2.3.1 Description

Permet à l'administrateur de visualiser toutes les plaintes sur une carte interactive avec possibilité de filtrer par statut, type, région ou période. Chaque plainte est représentée par un marqueur cliquable affichant ses détails complets incluant photos et informations géographiques.

### 2.3.2 Diagramme de séquence

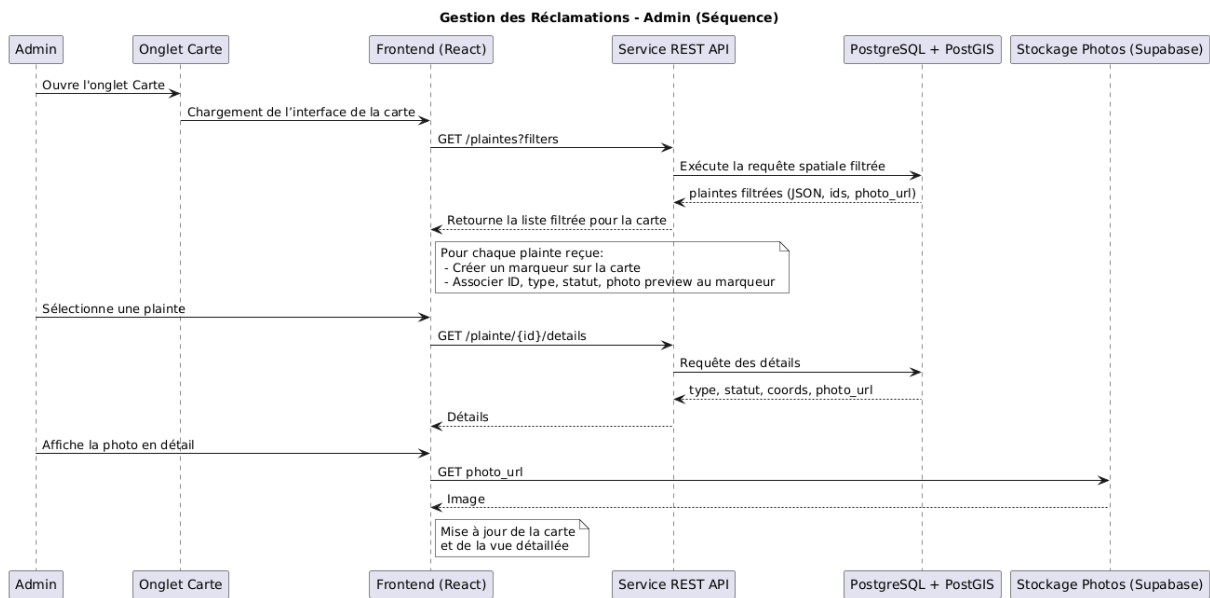


FIGURE 3 – Diagramme de séquence - Consultation et filtrage des plaintes sur la carte

### 2.3.3 Algorithme

---

**Algorithm 5** Consulter et filtrer les plaintes sur la carte

---

```
1: DÉBUT CONSULTEPLAINTESCART(filtres)
2:   Admin ouvre l'onglet Carte
3:   Frontend charge l'interface de la carte
4:   Frontend envoie GET /plaintes?filters=filtres
5:   Service REST API exécute requête spatiale filtrée
6:   PostgreSQL + PostGIS retourne plaintes filtrées (JSON, ids, coords, photo_url)
7:   API renvoie liste filtrée au Frontend
8:   POUR chaque plainte dans liste filtrée FAIRE
9:     Frontend crée marqueur sur carte avec :
10:      - ID plainte
11:      - Type de déchet
12:      - Statut (couleur marqueur)
13:      - Aperçu photo (si disponible)
14:   SI admin sélectionne une plainte (clic sur marqueur) ALORS
15:     Frontend envoie GET /plainte/{id}/details
16:     API récupère détails depuis PostgreSQL
17:     API retourne type, statut, coords, photo_url
18:     Frontend récupère photo complète depuis Stockage Supabase
19:     Afficher vue détaillée avec photo en grand
20:     Mettre à jour carte et vue détaillée
```

---

### 2.3.4 Filtres disponibles

- **Par statut** : En attente, En cours, Résolue, Annulée
- **Par région** : Région Nadhif spécifique ou Hors région
- **Par type de déchet** : Ménager, Recyclable, Encombrant, Dangereux
- **Par période** : Aujourd'hui, Cette semaine, Ce mois, Personnalisée

## 3 Module Citoyen

### 3.1 Suivi de Plainte

#### 3.1.1 Description

Permet au citoyen de suivre l'état de sa plainte en temps réel via son numéro de plainte. Le système affiche le statut actuel (En attente, En cours, Résolue) et la région concernée.

#### 3.1.2 Diagramme de séquence

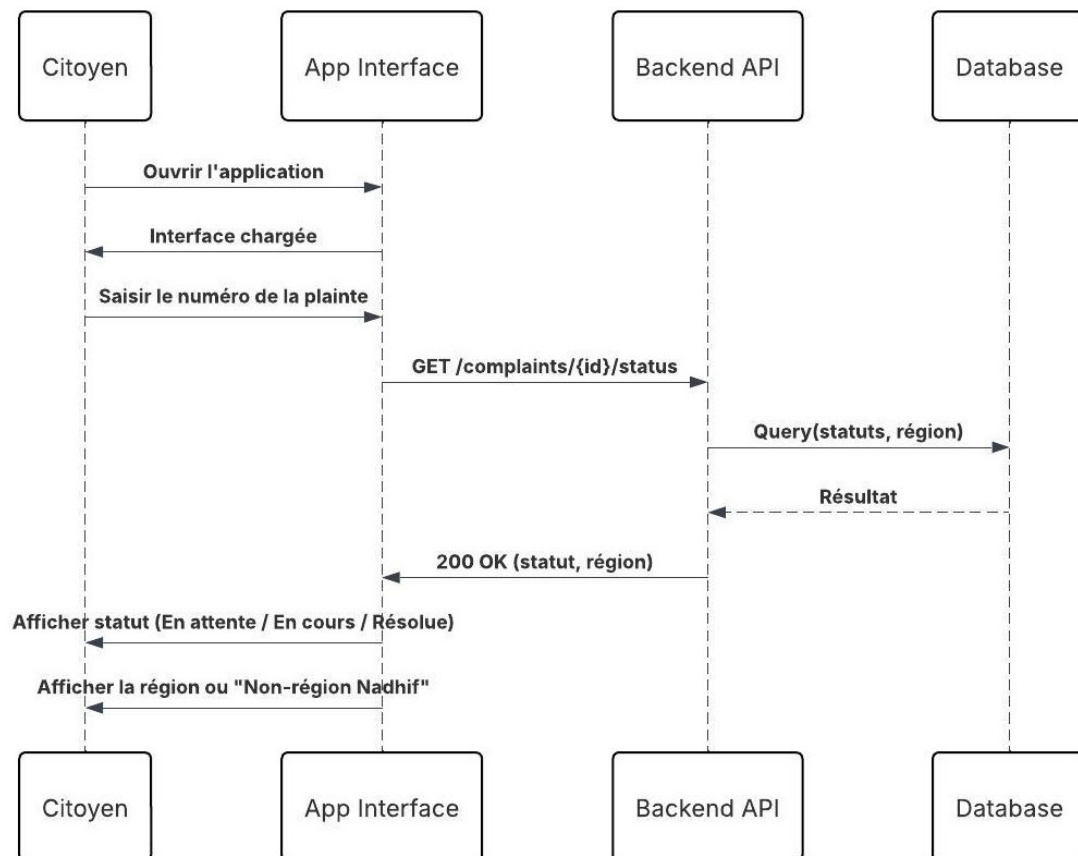


FIGURE 4 – Diagramme de séquence - Suivi de plainte

### 3.1.3 Algorithme

---

**Algorithm 6** Suivi de plainte

---

```
1: DÉBUT SUIVREPLAINTE(numeroPlainte)
2:   Citoyen ouvre l'application
3:   Interface se charge
4:   Citoyen saisit numeroPlainte
5:   Envoyer GET /complaints/{id}/status
6:   Backend interroge base de données
7:   SI plainte existe ALORS
8:     Récupérer statut et région
9:     Afficher statut (En attente / En cours / Résolue)
10:    SI région existe ALORS
11:      Afficher nom de la région
12:    SINON
13:      Afficher "Non-région Nadhif"
14:  SINON
15:    RETOURNER erreur "Plainte introuvable"
```

---

## 4 Module Administratif

### 4.1 Consultation des Plaintes

#### 4.1.1 Description

Permet à l'administrateur de consulter la liste complète des plaintes avec filtrage par région, statut, date, etc., et de visualiser les détails complets d'une plainte spécifique incluant photos et géolocalisation.

#### 4.1.2 Diagramme de séquence

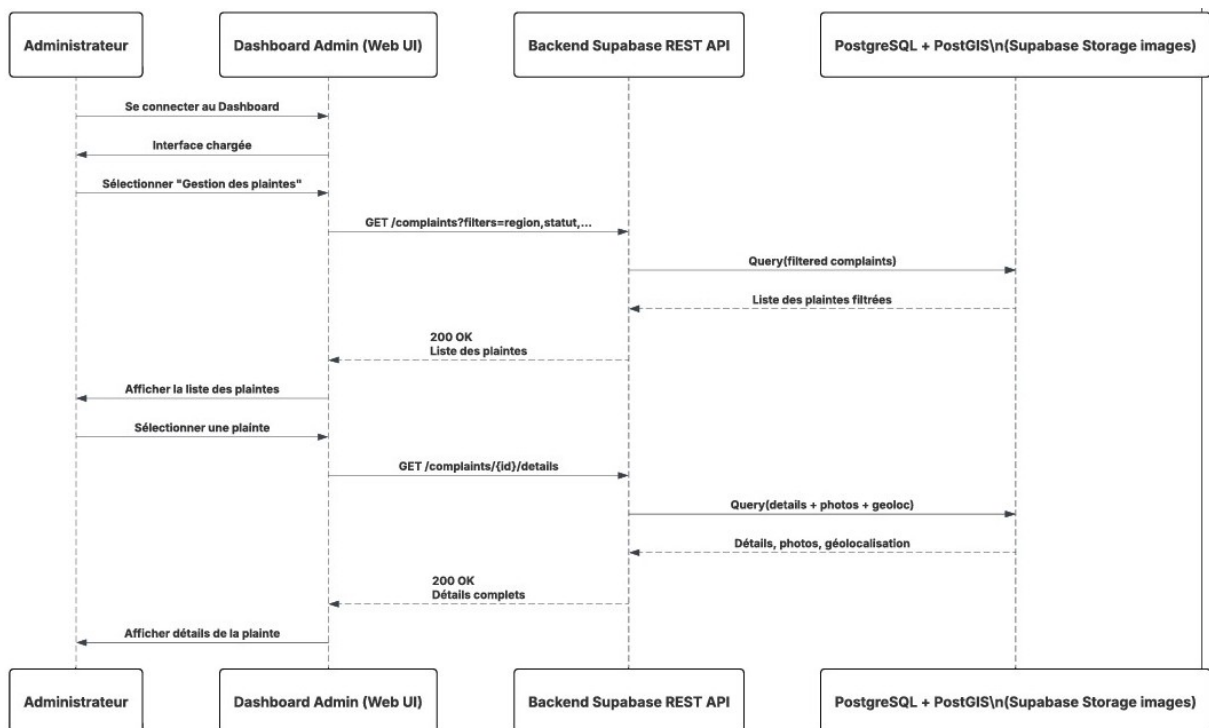


FIGURE 5 – Diagramme de séquence - Consultation des plaintes

### 4.1.3 Algorithme

---

**Algorithm 7** Consulter les plaintes

---

```
1: DÉBUT CONSULTEPLAINTES(filtres)
2:   Administrateur se connecte au Dashboard
3:   Interface se charge
4:   Administrateur sélectionne "Gestion des plaintes"
5:   Dashboard envoie GET /complaints?filters=filtres
6:   Backend interroge PostgreSQL avec filtres appliqués
7:   Backend retourne liste des plaintes filtrées
8:   Dashboard affiche la liste des plaintes
9:   SI administrateur sélectionne une plainte ALORS
10:     Dashboard envoie GET /complaints/{id}/details
11:     Backend récupère détails complets
12:     Backend retourne détails de la plainte
13:     Dashboard affiche détails complets
```

---

## 4.2 Modification du Statut des Plaintes

### 4.2.1 Description

Permet à l'administrateur de modifier le statut d'une plainte pour suivre son cycle de traitement (En attente → En cours → Résolue / Annulée).

### 4.2.2 Diagramme de séquence

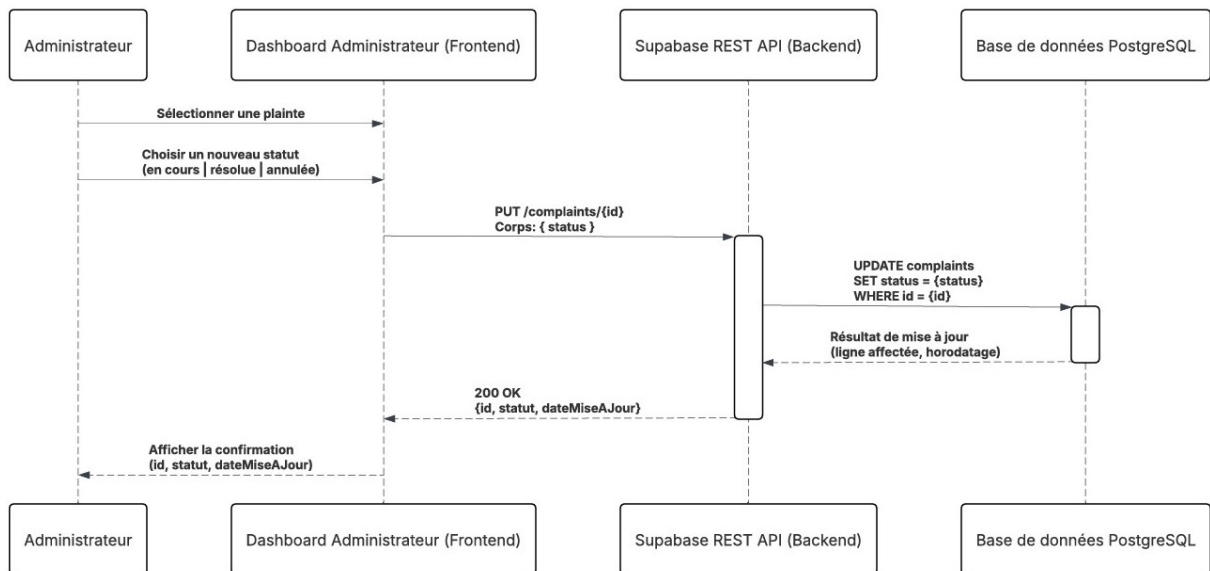


FIGURE 6 – Diagramme de séquence - Changement de statut

### 4.2.3 Algorithme

---

#### Algorithm 8 Changer le statut d'une plainte

---

- 1: **DÉBUT** CHANGERSTATUTPLAINTE(*idPlainte*, *nouveauStatut*)
  - 2:   Administrateur sélectionne plainte
  - 3:   Plainte affichée
  - 4:   Administrateur choisit nouveau statut
  - 5:   Dashboard envoie PUT /complaints/{id}
  - 6:   Backend exécute UPDATE complaints SET status
  - 7:   **SI** mise à jour réussie **ALORS**
  - 8:     Backend retourne 200 OK
  - 9:     Dashboard affiche confirmation
  - 10:  **SINON**
  - 11:   Dashboard affiche erreur
-

## 4.3 Export de Données

### 4.3.1 Description

Permet à l'administrateur d'exporter les plaintes filtrées au format Excel ou CSV.

### 4.3.2 Diagramme de séquence

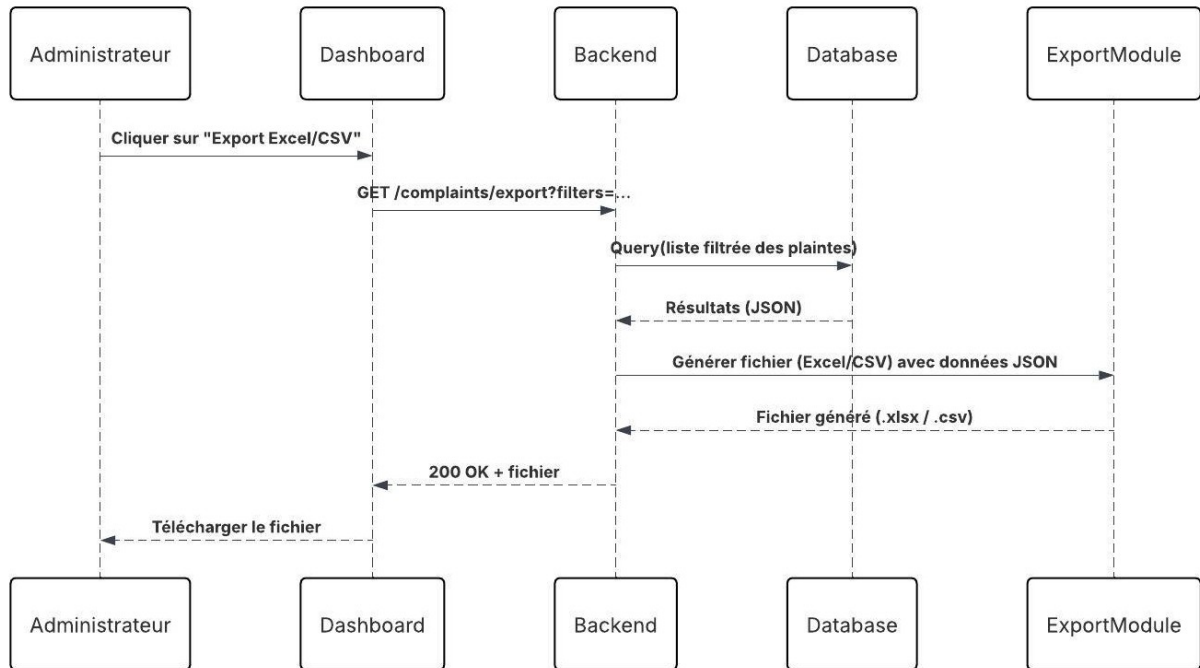


FIGURE 7 – Diagramme de séquence - Export de données

### 4.3.3 Algorithme

---

**Algorithm 9** Export de données

---

- 1: **DÉBUT** EXPORTERDONNEES(*filtres*, *format*)
  - 2: Administrateur clique sur "Export"
  - 3: Dashboard envoie GET /complaints/export ?filters
  - 4: Backend interroge base avec filtres
  - 5: Récupérer liste plaintes (JSON)
  - 6: Générer fichier selon *format* (.xlsx ou .csv)
  - 7: Backend retourne fichier généré
  - 8: Dashboard propose téléchargement
  - 9: Administrateur télécharge fichier
-



## 5 Gestion des Ressources

### 5.1 Gestion des Utilisateurs

#### 5.1.1 Description

Permet au Super Admin de gérer les comptes utilisateurs : création, modification, réinitialisation de mot de passe et activation/désactivation.

#### 5.1.2 Diagramme de séquence

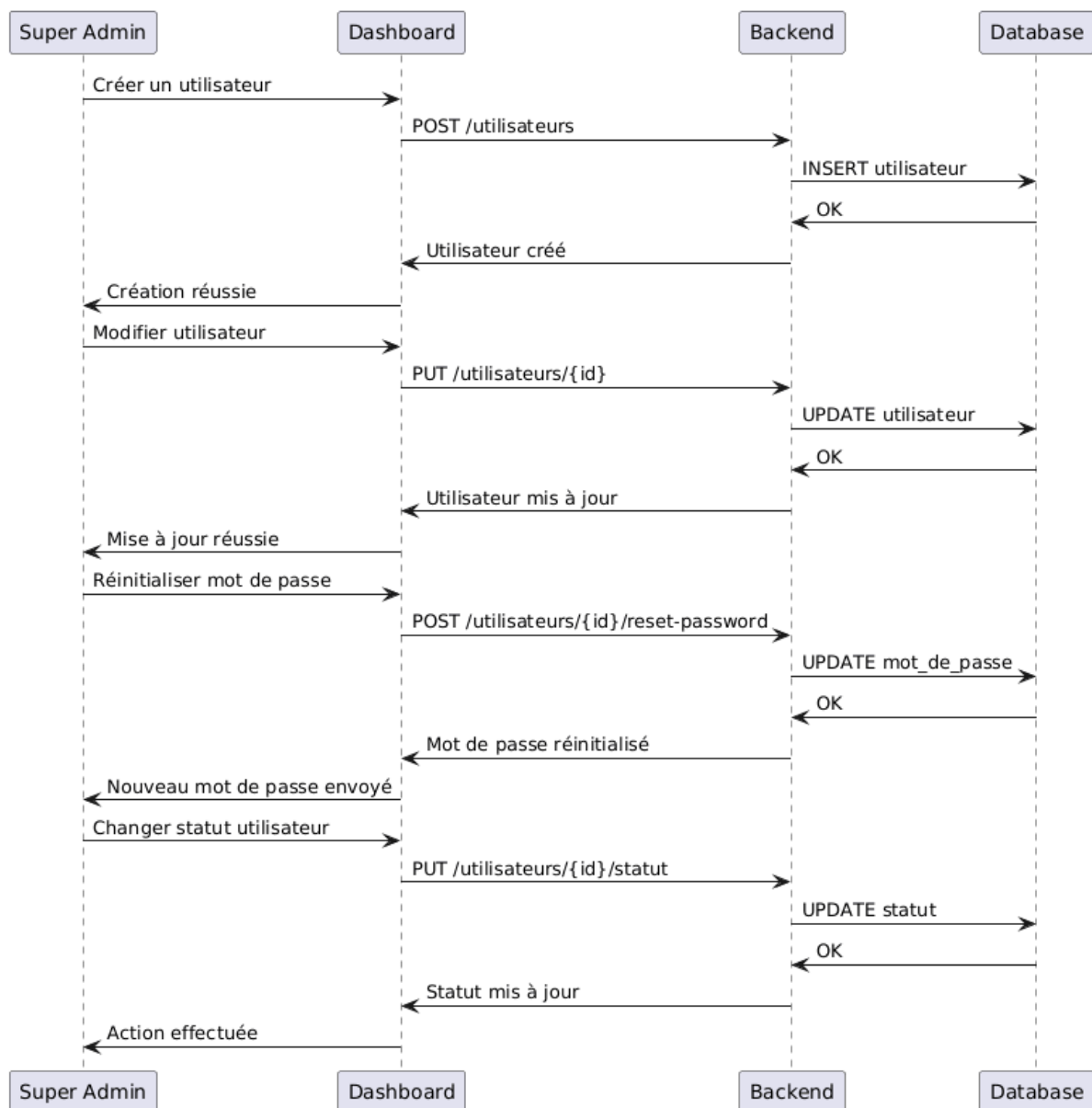


FIGURE 8 – Diagramme de séquence - Gestion des utilisateurs

### 5.1.3 Algorithme

---

**Algorithm 10** Créer un utilisateur

---

```
1: DÉBUT CREERUTILISATEUR(donneesUtilisateur)
2:   SI données invalides ALORS
3:     RETOURNER erreur "Données incorrectes"
4:   id ← Insérer donneesUtilisateur dans base
5:   SI id existe ALORS
6:     RETOURNER "Utilisateur créé avec succès"
7:   SINON
8:     RETOURNER erreur "Erreur lors de la création"
```

---

## 5.2 Gestion des Équipes

### 5.2.1 Description

Permet de créer et modifier des équipes de collecte, ainsi que de les assigner à des régions géographiques spécifiques.

### 5.2.2 Diagramme de séquence

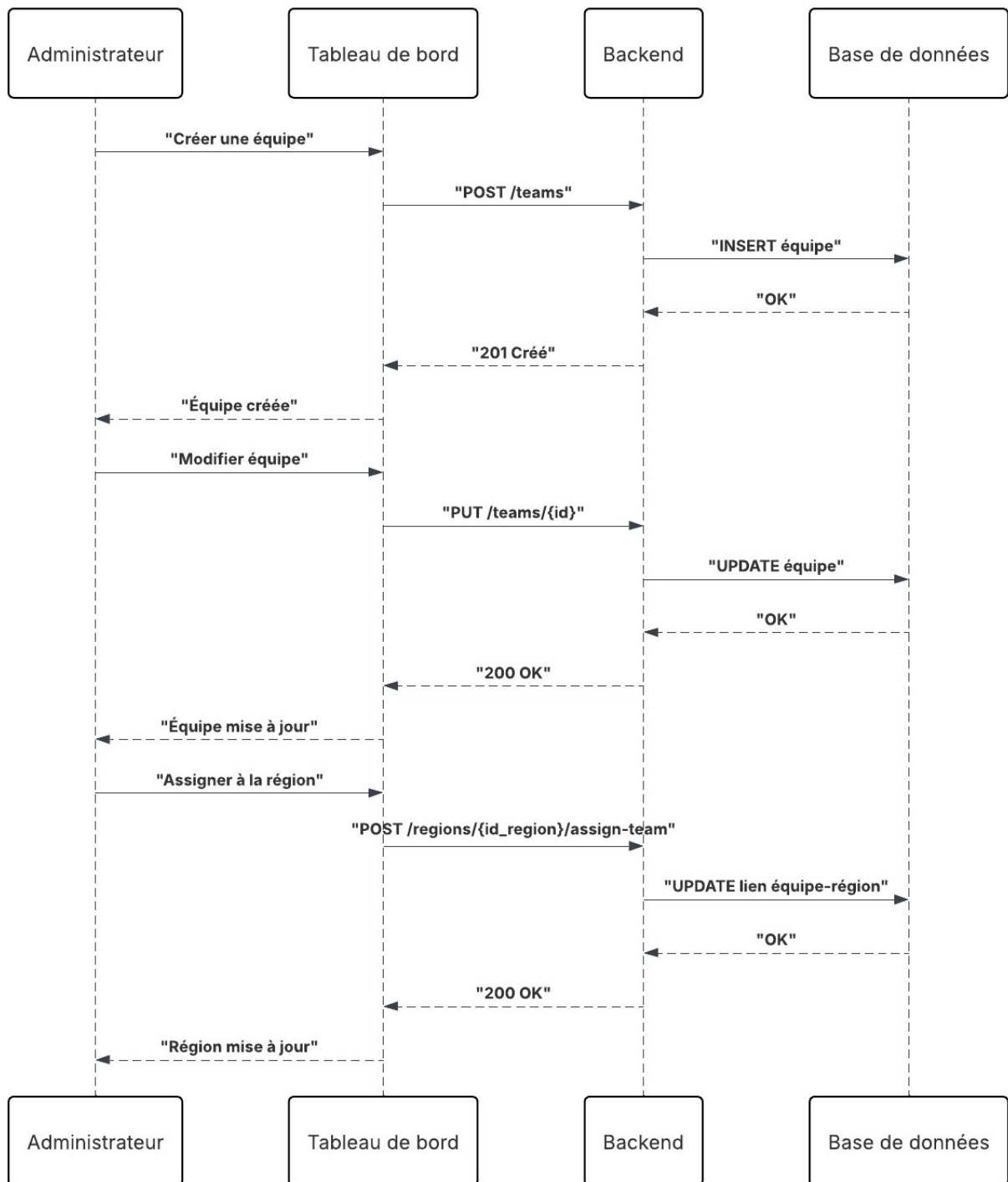


FIGURE 9 – Diagramme de séquence - Gestion des équipes

### 5.2.3 Algorithme

---

**Algorithm 11** Assigner équipe à région

---

```
1: DÉBUT ASSIGNEREQUIPEREGION(idEquipe, idRegion)  
2:   SI équipe n'existe pas ALORS  
3:     RETOURNER erreur "Équipe introuvable"  
4:   SI région n'existe pas ALORS  
5:     RETOURNER erreur "Région introuvable"  
6:   Mettre à jour liaison équipe-région  
7:   RETOURNER "Assignation réussie"
```

---

## 6 Module Analyse et Intelligence Artificielle

### 6.1 Validation IA des Images de Plaintes

#### 6.1.1 Description

Cette fonctionnalité utilise un modèle d'intelligence artificielle (CNN/ResNet) pour analyser automatiquement les photos soumises avec les plaintes. Le système détecte si l'image contient réellement des déchets, identifie leur type et attribue un score de confiance. Seules les images validées (score 70%) sont acceptées.

#### 6.1.2 Diagramme de séquence

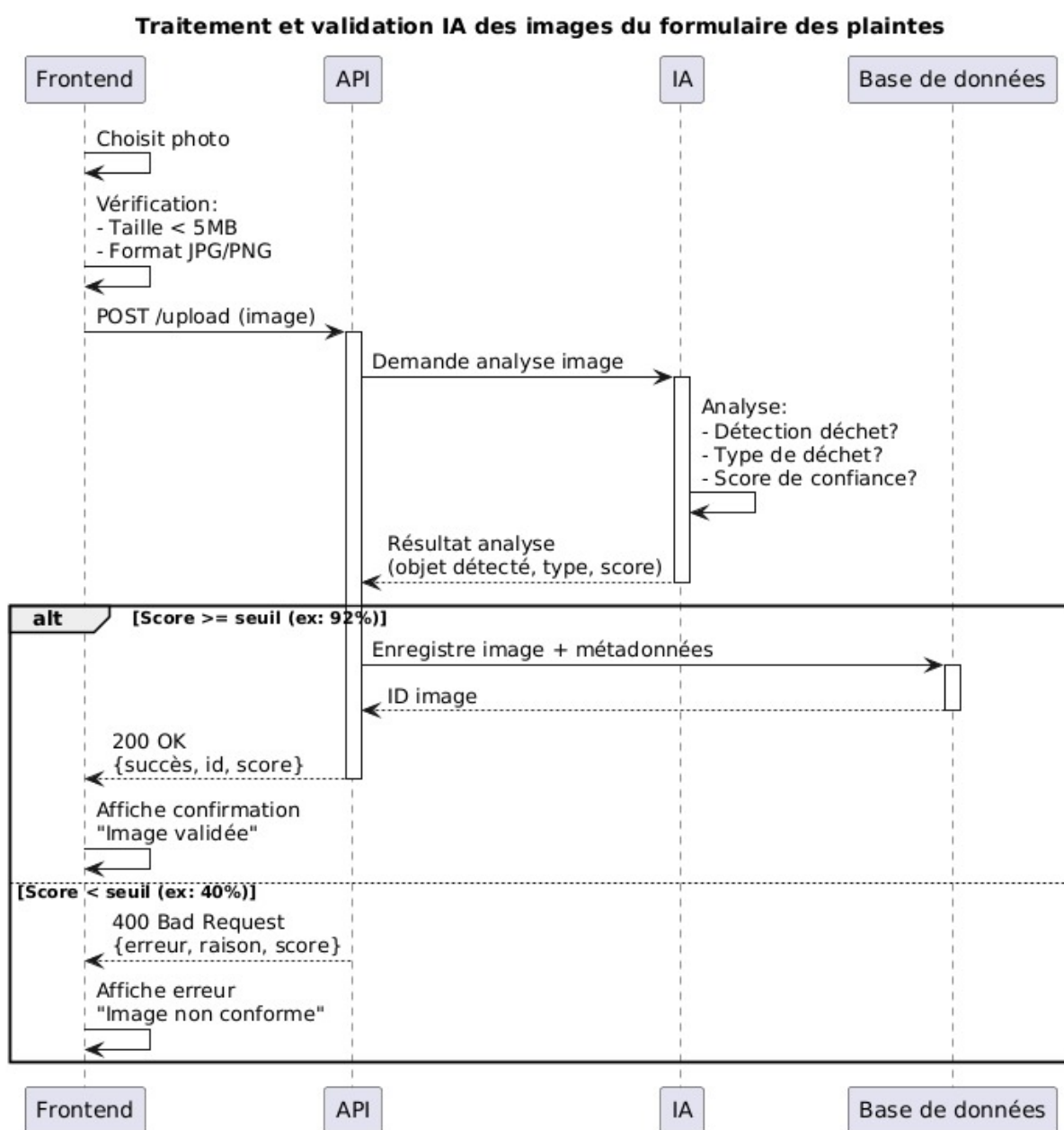


FIGURE 10 – Diagramme de séquence - Validation IA des images

### 6.1.3 Algorithme

---

**Algorithm 12** Validation IA des images de plaintes

---

```
1: DÉBUT VALIDERIMAGEIA(image, metadata)
2:   Utilisateur choisit photo
3:   Frontend vérifie format (JPG/PNG) et taille (max 5MB)
4:   SI validation client échoue ALORS
5:     RETOURNER erreur "Format ou taille invalide"
6:   Utilisateur soumet formulaire
7:   Frontend envoie POST /upload/image à API
8:   API vérifie type MIME et dimensions
9:   SI validation serveur échoue ALORS
10:    RETOURNER erreur 400 "Fichier invalide"
11:   API prépare image (redimensionne 224x224, normalise)
12:   API envoie au Modèle IA (CNN/ResNet)
13:   Modèle IA analyse et retourne :
14:     - is_waste (boolean)
15:     - waste_type (plastic, paper, metal, etc.)
16:     - confidence_score (0.0 à 1.0)
17:   SI is_waste == false OU confidence_score > 0.70 ALORS
18:     RETOURNER erreur 422 "Image rejetée - Pas de déchet détecté"
19:   API compresse image et génère UUID
20:   Envoyer image à Storage (Supabase)
21:   Storage retourne image_url et image_id
22:   Créer plainte avec métadonnées IA :
23:     - ai_waste_type, ai_confidence_score
24:   Insérer dans base de données
25:   RETOURNER succès avec plainte_id et détection IA
```

---

### 6.1.4 Types de déchets détectables

- Plastique (bouteilles, sacs, emballages)
- Papier et carton
- Métal (canettes, ferraille)
- Verre
- Déchets organiques
- Déchets mixtes/non triés

## 6.2 Statistiques et Graphiques

### 6.2.1 Description

Permet à l'administrateur de visualiser des statistiques agrégées sur les plaintes sous forme de graphiques interactifs. Les données peuvent être filtrées par période et exportées en image (PNG/PDF) pour reporting.

### 6.2.2 Diagramme de séquence

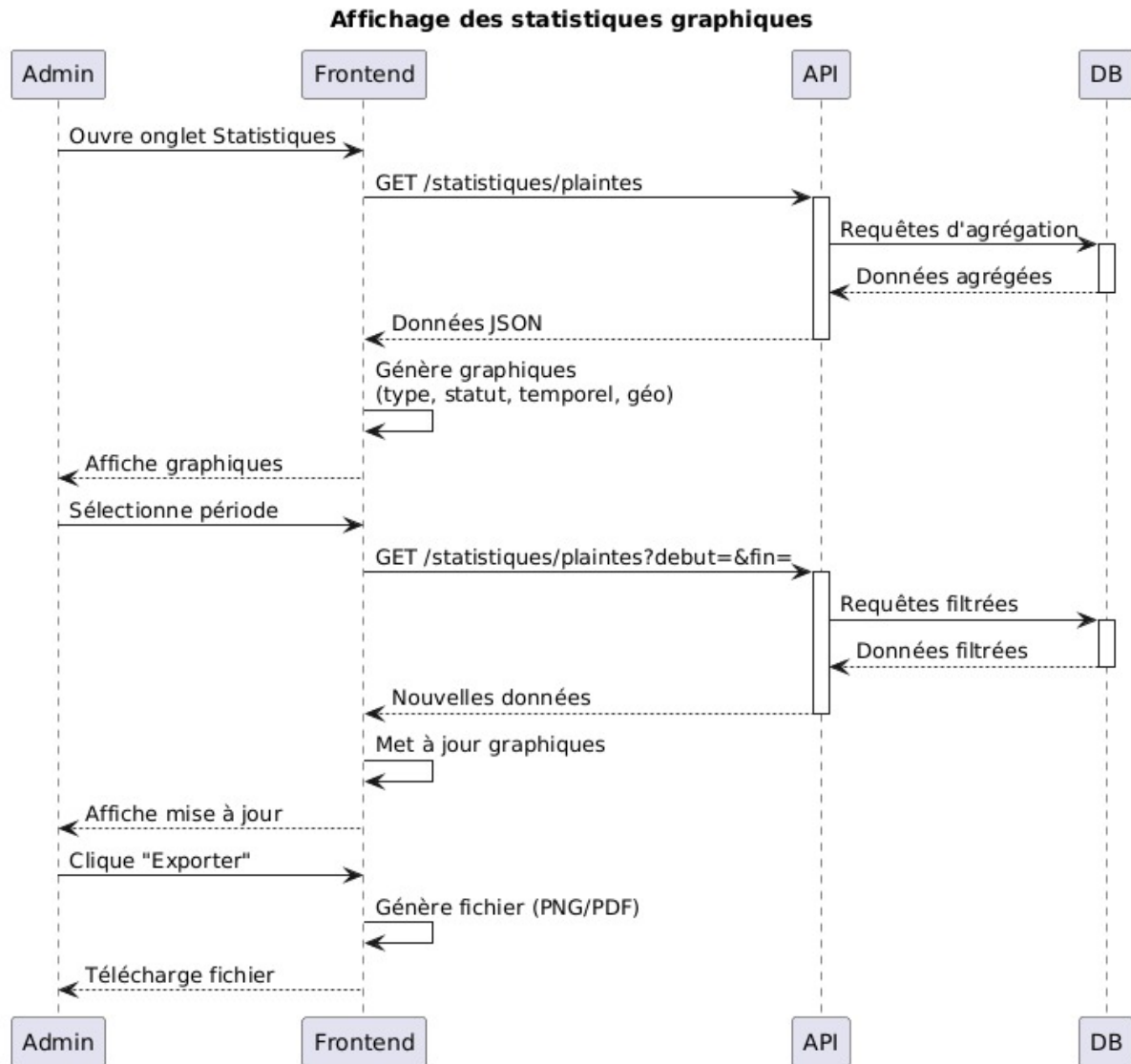


FIGURE 11 – Diagramme de séquence - Affichage des statistiques graphiques

### 6.2.3 Algorithme

---

**Algorithm 13** Affichage des statistiques graphiques

---

```
1: DÉBUT AFFICHERSTATISTIQUES(dateDebut, dateFin)
2:   Admin ouvre onglet Statistiques
3:   Frontend charge interface
4:   Frontend envoie GET /statistiques/plaintes à API
5:   API exécute requêtes d'agrégation sur base
6:   Base retourne données agrégées (JSON)
7:   API renvoie données à Frontend
8:   POUR chaque type de graphique FAIRE
9:     Frontend crée graphique :
10:      - Par type de déchet
11:      - Par statut (en attente, en cours, résolue)
12:      - Évolution temporelle
13:      - Répartition géographique
14:   Afficher tous les graphiques
15:   SI admin sélectionne période ALORS
16:     Frontend envoie GET /statistiques?debut=&fin=
17:     API exécute requêtes filtrées
18:     Base retourne données filtrées
19:     Frontend régénère graphiques
20:     Mettre à jour affichage
21:   SI admin clique "Exporter" ALORS
22:     Frontend génère image (PNG ou PDF)
23:     Télécharger fichier
```

---

### 6.2.4 Types de graphiques disponibles

- **Graphique circulaire** : Répartition par type de déchet
- **Graphique en barres** : Évolution par statut de traitement
- **Graphique linéaire** : Tendances temporelles (jour, semaine, mois)
- **Carte thermique** : Distribution géographique des plaintes
- **Graphique en colonnes** : Comparaison par région



## 7 Modèle de Données

### 7.1 Table utilisateurs

| Champ      | Type      | Description             |
|------------|-----------|-------------------------|
| id         | UUID      | Identifiant unique (PK) |
| email      | String    | Email (unique)          |
| role       | String    | super_admin / admin     |
| actif      | Boolean   | Statut actif/inactif    |
| created_at | Timestamp | Date de création        |

TABLE 1 – Structure de la table utilisateurs

### 7.2 Table wilayas

| Champ    | Type     | Description           |
|----------|----------|-----------------------|
| id       | Integer  | Identifiant (PK)      |
| nom      | String   | Nom de la wilaya      |
| code     | Integer  | Code wilaya (ex : 10) |
| geometry | Geometry | Polygone PostGIS      |

TABLE 2 – Structure de la table wilayas

### 7.3 Table regions

| Champ      | Type      | Description                |
|------------|-----------|----------------------------|
| id         | Integer   | Identifiant (PK)           |
| nom        | String    | Nom de la région           |
| type       | String    | résidentiel, industriel... |
| geometry   | Geometry  | Polygone PostGIS           |
| actif      | Boolean   | Actif/inactif              |
| created_by | UUID      | FK → utilisateurs.id       |
| created_at | Timestamp | Date de création           |

TABLE 3 – Structure de la table regions

### 7.4 Table equipes

| Champ       | Type    | Description         |
|-------------|---------|---------------------|
| id          | Integer | Identifiant (PK)    |
| nom         | String  | Nom de l'équipe     |
| chef_equipe | String  | Nom du chef         |
| telephone   | String  | Numéro de contact   |
| horaires    | String  | Horaires de travail |
| actif       | Boolean | Actif/inactif       |

TABLE 4 – Structure de la table equipes

## 7.5 Table region\_equipe

| Champ            | Type      | Description        |
|------------------|-----------|--------------------|
| id               | Integer   | Identifiant (PK)   |
| region_id        | Integer   | FK → regions.id    |
| equipe_id        | Integer   | FK → equipes.id    |
| actif            | Boolean   | Affectation active |
| date_affectation | Timestamp | Date d'affectation |

TABLE 5 – Table de liaison région-équipe

## 7.6 Table plaintes

| Champ           | Type      | Description       |
|-----------------|-----------|-------------------|
| id              | Integer   | Identifiant (PK)  |
| numero          | String    | Numéro unique     |
| description     | Text      | Description       |
| photo_url       | String    | URL photo         |
| latitude        | Float     | Latitude GPS      |
| longitude       | Float     | Longitude GPS     |
| type_dechet     | String    | Type de déchet    |
| statut          | String    | Statut traitement |
| region_id       | Integer   | FK → regions.id   |
| est_hors_nadhif | Boolean   | Hors région       |
| created_at      | Timestamp | Date de dépôt     |

TABLE 6 – Structure de la table plaintes

## 8 Architecture Système

### 8.1 Architecture 3-tiers

- **Couche Présentation** : Flutter (mobile), React (web)
- **Couche Logique** : Supabase (API REST + Realtime)
- **Couche Données** : PostgreSQL + PostGIS + Storage

### 8.2 Flux de données

1. **Citoyen** → Flutter → Supabase → PostgreSQL
2. **Admin** → React → Supabase → PostgreSQL
3. **PostGIS** : Calculs géospatiaux (ST\_Contains, ST\_Within)
4. **Leaflet** : Carte interactive + édition polygones

### 8.3 Sécurité

- Authentification JWT via Supabase Auth
- Row Level Security (RLS) sur PostgreSQL
- Communications HTTPS chiffrées
- Validation côté client et serveur

## 9 Conclusion

Le système Nadhif offre une solution complète pour la gestion des plaintes de collecte des déchets :

- Détection géographique automatique et précise
- Suivi transparent pour les citoyens
- Outils de gestion puissants pour les administrateurs
- Export et analyse des données
- Architecture moderne, sécurisée et évolutive

Cette conception permet à Epic Nadhif Bouira d'améliorer significativement la qualité du service de collecte et la satisfaction des citoyens.